Отчёт

Qt-команда · PyQt6

IFC Viewer — десктоп-приложение с 3D визуализацией

Смена стека

PyQt6

Каркас готов

Команда разработки

Смена стека: Qt C++ → PyQt6

Было: Qt C++

- Компилируемый C++ с Qt 6
- MOC-препроцессор, Q_OBJECT
- Долгая настройка CMake
- Высокая производительность
- Долгий цикл сборки
- IFC-движок с нуля на C++

Стало: PyQt6

- Python + PyQt6 биндинги
- Нет MOC — pyqtSignal
- pip install, быстрый старт
- Быстрые итерации
- Совместим с ifcopenshell
- Те же концепции Qt

Почему перешли — три причины

01 Мало времени

Реализовать IFC-движок с нуля на C++ за такое время нереально. Python + ifcopenshell даёт готовый парсер из коробки — без написания тысяч строк низкоуровневого кода.

02 IFC-команда тоже перешла на Python

Общий язык = прямая интеграция. Данные передаются через Python-структуры без FFI, биндингов и маршаллинга. Меньше точек стыка — меньше ошибок.

03 Концепции Qt полностью сохраняются

QWidget, сигналы/слоты, layouts, QMainWindow, QOpenGLWidget — всё то же самое. PyQt6 это Python-обёртка над тем же Qt. Изученная теория не потеряна.

День 1 — Среда, первое окно, архитектура PyQt6

Что изучили

`pip install PyQt6`	Установка и настройка окружения
`QApplication`	Точка входа, event loop, app.exec()
`QMainWindow`	Базовый класс главного окна
`pyqtSignal`	Вместо Q_OBJECT + MOC из C++
`@pyqtSlot`	Декоратор слота (необязателен, но явен)
`connect()`	Тот же синтаксис что в Qt C++

Ключевое отличие от C++

В C++ нужен MOC-препроцессор и макрос Q_OBJECT в каждом классе с сигналами. В PyQt6 — просто атрибут класса:

valueChanged = pyqtSignal(int)

Итог дня

Первое рабочее окно запущено. Счётчик с кнопкой и label: сигнал valueChanged(int) подключён к обновлению QLabel.

День 2 — Layouts и виджеты

Layouts

QVBoxLayout, QHBoxLayout, QGridLayout. Никаких фиксированных координат — всё через layout. addStretch() для пружин, setSpacing() и setContentsMargins() для отступов.

QSplitter

Разделитель между деревом и основной областью. Пользователь может двигать граница мышью. saveState() / restoreState() через QSettings.

QTreeWidget + QTableWidget

Заглушки для дерева IFC и таблицы свойств. Синтетические данные: проект → этаж → комната → стена. Клик по узлу → таблица обновляется.

Макет IFC Viewer

Собран полный визуальный макет без логики: дерево слева, пустая центральная область, панель свойств снизу. Все элементы масштабируются корректно.

PyQt6 vs Qt C++ — ключевые различия в синтаксисе

Концепция	Qt C++	PyQt6
Сигнал	`signals: void clicked();`	`clicked = pyqtSignal()`
Слот	`public slots: void onClicked();`	`@pyqtSlot() def on_clicked():`
Подключение	`connect(btn, &Btn::clicked, ...)`	`btn.clicked.connect(...)`
Emit	`emit valueChanged(42);`	`self.valueChanged.emit(42)`
Создание виджета	`auto *btn = new QPushButton(parent);`	`btn = QPushButton(parent)`
Запуск app	`return app.exec();`	`sys.exit(app.exec())`
Сборка	`cmake / qmake + компилятор`	`python main.py`

Итог недели 1

- Смена стека Qt C++ → PyQt6 выполнена без потерь
- Минимальный каркас IFC Viewer
- Макет интерфейса
- Концепции Qt усвоены — сигналы, slots, layouts